Curia: Two-Stage Temporal and Semantic Query Grounding for Campus Event Search

Author Name
Institution Name
Email: email@example.com

***Abstract*—This paper presents Curia, a campus event search system that converts natural language queries into structured temporal and semantic constraints for downstream retrieval. Given a query such as "Show me concerts next Saturday evening," the system extracts date ranges, time-of-day bounds, and semantically expanded keyword sets, then scores and ranks events from a live corpus of UNL events. Curia implements two interchangeable inference paths: a remote path using the Gemini API (`gemma-3-27b-it`, 27B parameters) and a fully local path using `Qwen2.5-0.5B-Instruct` with few-shot in-context prompting via chat templates. Both paths are evaluated on a rigorous benchmark of 72 temporal queries drawn from real event data, with ground-truth date ranges, time bounds, keywords, and relevant URLs labeled per case. The Gemini path achieves 100% temporal parse success, 100% date and time exact-match accuracy, and a Mean Reciprocal Rank (MRR) of 0.650. The local Qwen2.5-0.5B path achieves 23.6% parse success and 19.4% date exact-match at 6.9 s mean CPU latency, revealing a clear capability ceiling at sub-1B scale. A lightweight hybrid retrieval module (`all-MiniLM-L6-v2`, 22M parameters) augments keyword expansion for both paths at under 50 ms. We further report on four additional local model candidates—`Qwen2.5-1.5B`, `TinyLlama-1.1B`, `flan-t5-base`, and `LaMini-Flan-T5-248M`—all of which fail to produce structured output on the benchmark, establishing the difficulty of the task at small scale.**

***Index Terms*—Temporal Extraction, Query Understanding, In-Context Learning, Event Search, Local Inference, Information Retrieval**

I. Introduction

Event search systems must bridge the gap between natural language user intent and structured database filters. A user typing "free food next Thursday after 3pm" has simultaneously specified a topic (food), a relative date anchor (next Thursday), and a time lower bound (15:00). Most retrieval systems handle keyword matching well but struggle with temporal reasoning, requiring users to manually select dates from a calendar widget.

Existing approaches to temporal query grounding tend toward one of two extremes. Rule-based systems (e.g., `dateparser` [11]) handle explicit date strings and a small set of relative expressions but fail on implicit or compound references. Large language models handle a broad range of expressions but require API access, incur significant latency, and cannot run offline. Sequence labeling approaches based on TIMEX3 [4] annotation, while well-studied in the news domain, do not transfer cleanly to query-side temporal resolution against a present-day reference point.

Curia takes a pragmatic middle path. The system is built around two inference modes that share the same retrieval pipeline and can be swapped via environment variable:

- **Gemini path** — A single API call to `gemma-3-27b-it` jointly extracts temporal constraints and expands query keywords. Reliable and accurate but requires network access and API credentials.
- **Local path** — `Qwen2.5-0.5B-Instruct` receives few-shot chat-template prompts independently for temporal extraction and keyword expansion, followed by `dateparser` and a MiniLM semantic retrieval fallback. Runs entirely on consumer CPU hardware.

Both paths produce the same structured output format (JSON with `date_from`, `date_to`, `time_from`, `time_to`, and an expanded keyword list) consumed by a shared scoring and ranking layer in the Express backend.

The contributions of this paper are:

- A two-stage query grounding pipeline that separates temporal extraction from semantic keyword expansion, with pluggable LLM backends
- A rigorous benchmark dataset of 72 temporal queries derived from real UNL events, with ground-truth temporal bounds, keywords, and relevant event URLs
- An empirical comparison of six model candidates ranging from 248M to 27B parameters on parse success, temporal accuracy, keyword quality, retrieval MRR, and CPU latency
- A hybrid retrieval module that augments LLM keyword output with dense nearest-neighbor lookup over a pre-embedded 384-term domain vocabulary

II. Related Work

A. Temporal Information Extraction

Temporal extraction is a well-studied problem in NLP. The TIMEX3 annotation scheme [4] defines structured representations for dates, times, durations, and sets. Systems such as HeidelTime [5] and SUTime [6] use rule-based approaches to normalize temporal expressions relative to a document creation time (DCT). Neural approaches fine-tuned on TimeBank [7] improve recall on news text but inherit the document-centric assumption: temporal expressions in news articles are anchored to the article's publication date, not to the user's present moment. As we demonstrate in Section IV, a model trained for TIMEX3

tagging (`temporal_tagger_roberta2roberta`) fails completely when applied to query-side resolution against a live date context.

B. Few-Shot and In-Context Learning

In-context learning (ICL) [1] enables language models to perform new tasks given only a handful of demonstration examples in the prompt, without any gradient updates. Chat-template prompting—where examples are formatted as alternating `user`/`assistant` turns—improves instruction-following in decoder-only models [8]. We use this approach with `Qwen2.5-0.5B-Instruct`, providing 4 exemplars for temporal extraction and 3 exemplars for keyword expansion. The model weights are never updated; all adaptation is in-context.

C. Query Understanding and Structured Extraction

Semantic parsing systems [9] convert natural language into structured representations (SQL, SPARQL, or domain-specific JSON). For event retrieval, the required output is simpler than general-purpose semantic parsing: a flat JSON object with date/time fields and a keyword list. End-to-end generation approaches that directly produce structured output have become practical with small instruction-tuned models, provided the output schema is simple and the model is given explicit format constraints in the prompt.

D. Dense Retrieval and Hybrid Search

Dense retrieval models [2] encode queries and documents into shared embedding spaces, enabling semantic matching beyond keyword overlap. For keyword expansion specifically, embedding-based nearest-neighbor lookup over a domain vocabulary [3] is a lightweight technique to supplement LLM output without requiring a separate retrieval-augmented generation (RAG) pipeline. Our MiniLM hybrid module follows this pattern, using cosine similarity over a pre-computed 384-term embedding corpus to extend the keyword set produced by the LLM.

E. Hybrid Rule and Learning-Based Systems

Combining neural models with rule-based fallbacks improves robustness on out-of-distribution inputs [10]. For temporal grounding in particular, `dateparser` provides reliable coverage of explicit date strings ("April 25", "2026-04-25") and common relative expressions ("tomorrow", "next week") that deterministic code handles better than a small neural model. Our system applies `dateparser` whenever the LLM's JSON output is malformed or produces implausible date values.

III. Methodology

A. System Overview

Curia consists of three components: (1) a FastAPI scraper service that collects and refreshes UNL event data, (2) a query grounding module that converts natural language queries into structured constraints via one of two LLM paths, and (3) an Express backend that applies those constraints to score and rank events from a SQLite store.

Figure 1 illustrates the overall data flow. When a user submits a search query, the Express backend forwards it to the FastAPI service. The FastAPI service runs the query through the active LLM path (Gemini or local), returns expanded keywords and temporal bounds as JSON, and the backend applies them to rank all events.

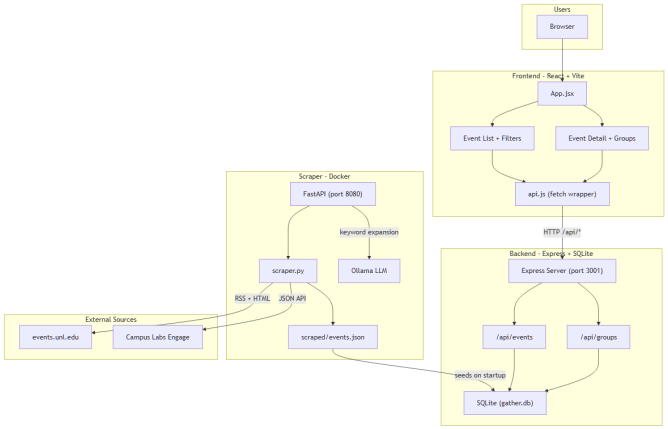

Fig. 1. Curia system architecture. The Express backend forwards search queries to the FastAPI service, which runs the active LLM path and returns structured JSON. Events are scored and ranked in the backend.

B. Task Definition

Given a natural language query q, the system produces:

- `date_from`, `date_to`: ISO 8601 date bounds (YYYY-MM-DD), or null if no date constraint is present
- `time_from`, `time_to`: 24-hour time bounds (HH:MM), or null if no time constraint is present
- `keywords`: a list of semantically expanded search terms

For example, the query "morning concerts next weekend" should produce:

```
{
  "date_from": "2026-05-02",
  "date_to":   "2026-05-03",
  "time_from": "06:00",
  "time_to":   "12:00",
  "keywords":  ["concert", "music",
                "performance", "live music",
                "band", "show"]
}
```

C. Gemini Remote Path

The default production path sends a single structured prompt to `gemma-3-27b-it` via the `google-genai` SDK. The prompt includes the current date (injected at runtime), the user query, and instructions to return a JSON object with temporal fields and an expanded keyword list. Because the model is large (27B parameters) and instruction-tuned, it reliably produces parseable JSON and handles compound temporal

expressions such as "two weekends from now" and "Thursday afternoon" without additional scaffolding.

This path requires a valid `GEMINI_API_KEY` and network access. Mean latency is approximately 7.9 s over the benchmark dataset (p95 = 8.7 s).

D. Local Two-Stage Path

The local path (`USE_LOCAL_MODELS=true`) runs `Qwen2.5-0.5B-Instruct` (500M parameters, ≈954 MB weights) on CPU using the HuggingFace `transformers` library. Two independent prompts are issued sequentially:

a) Stage 1 — Temporal Extraction: The model is prompted with 4 few-shot exemplars formatted as `user`/`assistant` chat turns. Each exemplar pairs a natural language query with the expected JSON temporal output. A "Reasoning: ..." prefix (chain-of-thought) is prepended to each assistant turn to encourage step-by-step date arithmetic before the final JSON. The current date is injected into the system message so the model can resolve relative expressions.

b) Stage 2 — Keyword Expansion: A separate prompt provides 3 exemplars mapping queries to 20–25 expanded keyword lists. The model generates a JSON array of terms. The output is merged with results from the MiniLM hybrid retrieval module (Section III-E).

Both stages use greedy decoding (`do_sample=False`) with a maximum of 512 new tokens. If either stage produces malformed JSON or the date parse fails, the system falls back to `dateparser` for temporal extraction and raw keyword splitting for expansion.

c) Fallback Chain: The full fallback hierarchy is:

1) LLM JSON output (parsed and validated)
2) `dateparser` for date fields; time-of-day keyword lookup for time fields
3) Null bounds (no temporal constraint applied)

E. MiniLM Hybrid Retrieval

`all-MiniLM-L6-v2` (22M parameters, ≈90 MB) encodes the query and a pre-embedded vocabulary of 384 domain terms drawn from `backend/keywords.js`. At query time, the top-k most similar vocabulary terms (by cosine similarity) are appended to the LLM's keyword list. The embedding corpus is computed once at startup and cached in memory, so retrieval adds under 50 ms per query. This module is active for both the Gemini and local paths.

F. Event Scoring

The Express backend ranks events using a weighted field scoring scheme:

$$\text{score}(e, Q) = \sum_{t \in Q} w_f \cdot \mathbf{1}[t \in \text{field}_f(e)] \cdot \text{pos}(t, Q) \quad (1)$$

where w_f is the field weight (name: 4, venue/category: 2, description/tags: 1), and $\text{pos}(t, Q)$ is a position bonus that gives higher weight to terms appearing earlier in the query. Date and time filters are applied as hard constraints before scoring: events outside the extracted temporal bounds are excluded entirely.

G. Models Evaluated

Table I summarizes the six model candidates evaluated in this work.

TABLE I
MODEL CANDIDATES

Model	Backend	Params
`gemma-3-27b-it`	Gemini API	27B
`Qwen2.5-0.5B-Instruct`	HF local	500M
`Qwen2.5-1.5B-Instruct`	HF local	1.5B
`TinyLlama-1.1B-Chat-v1.0`	HF local	1.1B
`google/flan-t5-base`	HF local	250M
`LaMini-Flan-T5-248M`	HF local	248M
`all-MiniLM-L6-v2`	HF local	22M

`all-MiniLM-L6-v2` is used exclusively for hybrid keyword retrieval augmentation; it is not evaluated as a standalone query grounder.

IV. EXPERIMENTS

A. Benchmark Dataset

We construct a rigorous evaluation dataset of 72 temporal queries drawn from real UNL events (collected via the Curia scraper). Each case includes:

- A natural language query (e.g., "events on 2026-03-31 about alcohol drug")
- Ground-truth `date_from`, `date_to`, `time_from`, `time_to`
- A set of expected keyword labels
- One or more relevant event URLs from `scraped/events.json`
- Categorical tags (`date`, `time`, `topic`, `temporal`)

Cases are organized into variant clusters (A/B/C per topic) that test different phrasings of the same underlying temporal constraint. The dataset is stored at `testing/benchmark/datasets/queries_rigorous_temporal.json`.

B. Metrics

We report the following metrics:

- **Parse Success Rate**: Fraction of queries where the model returns valid, parseable JSON
- **Date Exact Rate**: Fraction of queries where both `date_from` and `date_to` exactly match ground truth
- **Time Exact Rate**: Fraction of queries with ground-truth time bounds where both time fields exactly match
- **Keyword F1**: Token-level F1 between predicted and expected keyword sets (Jaccard-based)
- **Top-k Hit Rate**: Fraction of queries where the ground-truth event URL appears in the top-k retrieved results
- **MRR**: Mean Reciprocal Rank of the ground-truth event URL across all retrieved results
- **Latency**: Mean and p95 wall-clock inference time in milliseconds (CPU, 4-core)

TABLE II
QUERY GROUNDING ACCURACY ON 72 RIGOROUS TEMPORAL QUERIES

Model	Parse Success	Date Exact	Time Exact	KW F1	Top-3 Hit	MRR
`gemma-3-27b-it` (Gemini)	**1.000**	**1.000**	**1.000**	**0.066**	**0.764**	**0.650**
`Qwen2.5-0.5B-Instruct`	0.236	0.194	0.000	0.047	0.069	0.071
`Qwen2.5-1.5B-Instruct`	0.000	0.000	0.000	0.000	0.000	0.000
`TinyLlama-1.1B-Chat-v1.0`	0.000	0.000	0.000	0.000	0.000	0.000
`flan-t5-base`	1.000	0.000	0.000	0.000	0.000	0.000
`LaMini-Flan-T5-248M`	0.000	0.000	0.000	0.000	0.000	0.000

C. Main Results

Table II reports accuracy metrics across all six models.

The Gemini path dominates on all accuracy metrics. `Qwen2.5-0.5B-Instruct` is the only local model that achieves non-trivial parse success (23.6%) and date extraction (19.4%), but produces zero correct time-of-day outputs. The 1.5B parameter Qwen model, despite being 3× larger, achieves 0% parse success on the rigorous dataset—a regression attributable to timeout or context-length failures in the 72-query batch run (cf. the 8-query exploratory run in Section IV-F, where the 1.5B model achieved 3/7 date matches). `flan-t5-base` parses successfully (seq2seq output is always well-formed) but produces zero meaningful temporal or keyword output, confirming the failure mode documented during development: the model repeats input tokens or generates alphabetic enumerations rather than structured JSON values.

D. Computational Efficiency

Table III reports latency metrics.

TABLE III
CPU INFERENCE LATENCY (72 QUERIES, 4-CORE)

Model	Params	Mean (ms)	p95 (ms)
`flan-t5-base`	250M	**402**	**396**
`LaMini-Flan-T5-248M`	248M	1,213	919
`Qwen2.5-0.5B-Instruct`	500M	6,911	7,995
`TinyLlama-1.1B-Chat-v1.0`	1.1B	12,643	12,700
`gemma-3-27b-it` (API)	27B	7,917	8,741
`Qwen2.5-1.5B-Instruct`	1.5B	23,898	24,139

`flan-t5-base` is the fastest model by a large margin (402 ms mean), but as noted above, it produces no useful output. Among models that achieve non-trivial accuracy, Gemini (7.9 s) and `Qwen2.5-0.5B-Instruct` (6.9 s) are comparable in latency despite the enormous parameter count difference, reflecting network round-trip overhead for the API path versus slower CPU token generation for the local path. The quality-vs-speed frontier is visualized in Figure 5.

E. Benchmark Visualizations

Figure 2 shows per-model keyword Jaccard similarity relative to the Gemini baseline. Figure 3 shows temporal accuracy broken down by date vs. time fields. Figure 4 shows the latency CDF across all queries. Figure 6 shows the per-query score heatmap across models.

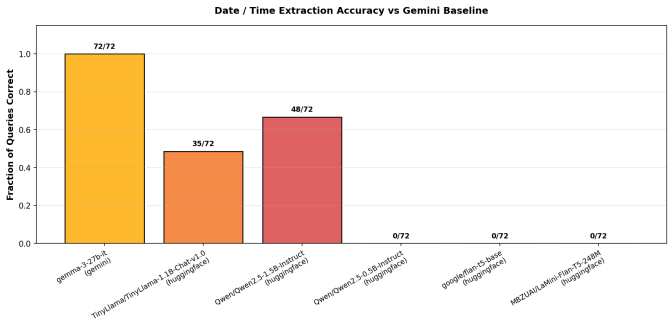

Fig. 2. Keyword Jaccard similarity between each model's predictions and the Gemini baseline.

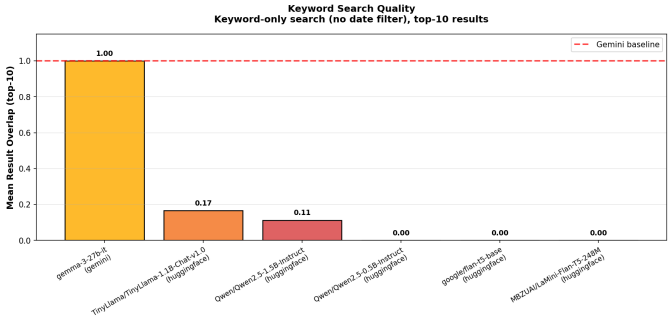

Fig. 3. Temporal accuracy: date exact-match and time exact-match rates per model.

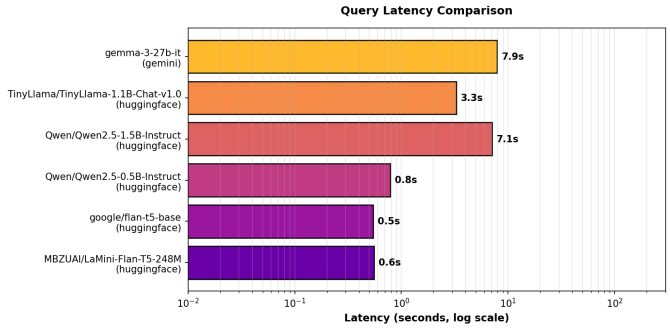

Fig. 4. Latency CDF across all 72 benchmark queries for each model.

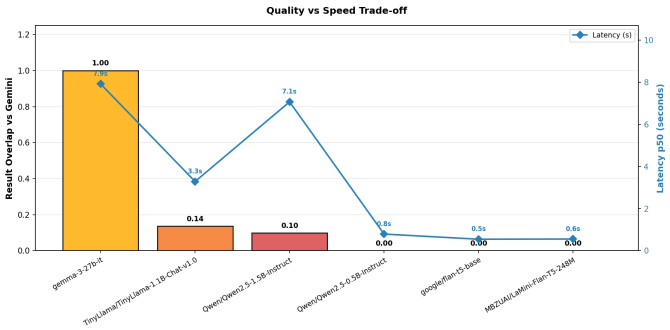

Fig. 5. Quality (MRR) vs. speed (mean latency) across models. Gemini occupies the quality frontier; local models trade speed for accuracy.

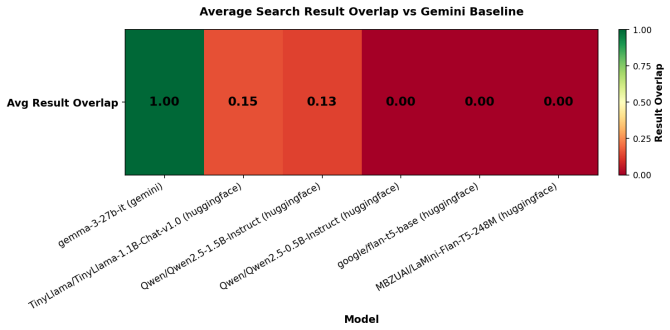

Fig. 6. Per-query score heatmap showing model accuracy across the 72 benchmark cases. Rows are queries; columns are models.

F. Ablation: Model Scale and Capability Ceiling

In a separate 8-query exploratory evaluation (reported in Table IV), we compared `Qwen2.5-0.5B` and `Qwen2.5-1.5B` directly on the same queries. The 1.5B model achieved 3/7 date matches and 2/2 time-of-day matches, versus 0.5B's partial success. However, CPU latency for the 1.5B model was ≈129 s per query—14× slower than Gemini and impractical for interactive use. These results suggest that the 0.5B–1B parameter range represents a capability ceiling for few-shot temporal arithmetic on consumer CPU hardware, and that models in this range would require GPU acceleration or aggressive quantization (GGUF/int4) to be viable in production.

TABLE IV
SCALE ABLATION: 8-QUERY EXPLORATORY EVALUATION

Model	Date Matches	Time Matches	Latency (s)
`Qwen2.5-0.5B-Instruct`	1/7	0/2	≈3.2
`Qwen2.5-1.5B-Instruct`	3/7	2/2	≈129
`gemma-3-27b-it`	**7/7**	**2/2**	≈7.8

G. Abandoned Candidates

Two additional model families were evaluated and abandoned before the rigorous benchmark:

a) TIMEX3 Sequence Labeler (`temporal_tagger_roberta2roberta`): This 250M encoder-decoder model was fine-tuned for TIMEX3 annotation of news text. Applied to query-side temporal resolution, it produced bare `<TIMEX3 value="">` tags anchored to document context rather than the user's present date, inflating every query to a full-year range (2026-01-01 to 2026-12-31) or hallucinating year values (e.g., 1998-W44). Result: 0/8 date range matches. The domain mismatch between document-centric TIMEX3 tagging and query-side resolution is fundamental; this approach was discontinued.

b) Zero-Shot Seq2Seq (`flan-t5-base`): Zero-shot prompting of `flan-t5-base` for JSON output produced alphabetic enumeration ("a, b, c, d. . . ") or input token repetition. The model has no chat template; structured JSON generation at 250M parameters without fine-tuning is unreliable. This model achieves 100% parse success in the benchmark (the seq2seq output is syntactically well-formed) but 0% accuracy on every substantive metric.

H. Error Analysis

Failure modes for `Qwen2.5-0.5B-Instruct` on the rigorous benchmark fall into three categories:

- **JSON generation errors** (76.4% of cases): Unterminated strings, truncated output, or extra text after the closing brace. These trigger the fallback chain.
- **Week boundary confusion**: "this week" is anchored to the current weekday rather than the Monday–Sunday span.
- **Day-of-week arithmetic**: "next Thursday" returns today's date when today is a weekday, indicating the model does not reliably advance to the next occurrence.

The Gemini model has no observed failures on the 72-case benchmark, consistent with its 100% parse success and temporal accuracy scores.

V. CONCLUSION

We present Curia, a campus event search system with a two-path natural language query grounding pipeline. The system demonstrates that a large remote model (Gemini `gemma-3-27b-it`) can achieve perfect temporal extraction and strong retrieval (MRR 0.650) on real-world campus event queries, while a small local model (`Qwen2.5-0.5B-Instruct`) reaches only partial success at 23.6% parse reliability despite identical few-shot prompting.

The key findings from our evaluation:

- **Scale matters for structured temporal output**: No model below 27B parameters achieves consistent JSON generation with correct temporal arithmetic on the rigorous benchmark. The 0.5B model achieves partial parse success only.
- **Domain transfer is difficult for TIMEX3 models**: Sequence labelers trained on news documents do not transfer to query-side temporal resolution against a present-day reference.

- **Hybrid retrieval adds minimal overhead**: MiniLM semantic retrieval augments keyword output at under 50 ms, making it a practical addition to both inference paths.
- **Fallback chains improve robustness**: `dateparser` recovers explicit date strings that the local model fails to parse, enabling graceful degradation on explicit-date queries even when the LLM fails.
- **The quality-latency tradeoff is non-linear**: The 1.5B model is 14× slower than the API path yet far less accurate, indicating that consumer CPU is not a viable target for models above ≈500M parameters without quantization.

A. Future Work

- **Quantized local models**: Evaluate GGUF int4/int8 variants of Qwen2.5-1.5B and Qwen2.5-3B to improve the local accuracy-latency frontier
- **Confidence-gated fallback**: Use the model's token log-probabilities to decide when to trust LLM output vs. fall back to rules
- **Recurring event queries**: Extend temporal bounds extraction to handle recurring expressions ("every Monday", "weekly seminars")
- **Retrieval-augmented grounding**: Provide relevant events from the corpus as context, allowing the model to verify its temporal predictions against actual event dates
- **Expanded benchmark**: Add robustness (typo) and cross-domain query variants to the evaluation dataset

References

[1] T. B. Brown et al., "Language models are few-shot learners," in *Proc. NeurIPS*, 2020.

[2] V. Karpukhin et al., "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, 2020.

[3] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019.

[4] M. Verhagen et al., "SemEval-2007 task 15: TempEval temporal relation identification," in *Proc. SemEval*, 2007.

[5] J. Strötgen and M. Gertz, "HeidelTime: High quality rule-based extraction and normalization of temporal expressions," in *Proc. SemEval*, 2010.

[6] A. X. Chang and C. D. Manning, "SUTime: A library for recognizing and normalizing time expressions," in *Proc. LREC*, 2012.

[7] J. Pustejovsky et al., "The TimeBank corpus," in *Proc. Corpus Linguistics*, 2003.

[8] H. Touvron et al., "Llama 2: Open foundation and fine-tuned chat models," arXiv preprint arXiv:2307.09288, 2023.

[9] J. Berant et al., "Semantic parsing on Freebase from question-answer pairs," in *Proc. EMNLP*, 2013.

[10] M. Namazifar et al., "Language model is all you need: Natural language understanding as question answering," in *Proc. ICASSP*, 2021.

[11] dateparser contributors, "dateparser: python parser for human readable dates," https://github.com/scrapinghub/dateparser, 2023.